

Lab: Building Your First Skill

Module: Day 3 - Claude Code Advanced

Duration: 30 minutes

Difficulty: Intermediate

Overview

In this lab, you will build a complete skill from scratch — a reusable package of instructions, templates, and scripts that Claude loads on demand. By the end, you'll have a working skill that produces consistent, high-quality output for a real business task.

Objectives

After completing this lab, you will be able to:

- Create a SKILL.md with proper YAML frontmatter
- Write effective trigger descriptions
- Control invocation (manual-only vs auto-load)
- Use \$ARGUMENTS for parameterized skills
- Add supporting files (templates, scripts)
- Install skills from the marketplace

Prerequisites

- Claude Code installed and authenticated
- Familiarity with custom commands (Lab: Custom Commands)
- Sample project from earlier labs

What You'll Build

A **meeting-notes** skill that: - Generates structured meeting summaries from raw notes - Uses a consistent template every time -

Includes action items with owners and due dates - Outputs a professional markdown document

Exercise 1: Create a Basic Skill

Duration: 10 minutes

Task 1: Set Up the Skill Directory

1. Navigate to your project:

```
bash cd labs/sample-project
```

1. Create the skill directory:

```
bash mkdir -p .claude/skills/meeting-notes
```

1. Create the SKILL.md file:

```
bash touch .claude/skills/meeting-notes/SKILL.md
```

Task 2: Write the SKILL.md

Open `.claude/skills/meeting-notes/SKILL.md` and add:

```
---
name: meeting-notes
description: Transforms raw meeting notes into structured summaries w
argument-hint: [paste raw notes or describe the meeting]
---

# Meeting Notes Formatter

## Instructions

When given raw meeting notes or a meeting description:

1. Extract key information:
   - Meeting date, attendees, and purpose
```

- Main discussion points (3-5 bullets max)
 - Decisions made (with rationale)
 - Action items (who, what, by when)
2. **Format as structured markdown:**
- Use the template in template.md
 - Keep summaries concise – one paragraph per topic
 - Bold action item owners
 - Include "Next Meeting" section if mentioned
3. **Quality checks:**
- Every action item must have an owner AND a due date
 - If a due date wasn't mentioned, flag it: "⚠ No due date specified"
 - If attendees weren't listed, ask before generating

Examples

Input: "Met with Sarah and Tom about the API redesign. Tom will handle the auth module by Friday. Sarah is researching rate limiting options. We decided REST over GraphQL for simplicity. Next sync Thursday 2pm."

Output: See template.md for the expected format.

Task 3: Test the Basic Skill

1. In Claude Code, type:

```
/meeting-notes Met with Sarah and Tom about the API redesign. Tom will handle the auth module by Friday. Sarah is researching rate limiting options. We decided REST over GraphQL for simplicity. Next sync Thursday 2pm.
```

1. Verify Claude produces a structured summary with:
2. ☐ Attendees listed
3. ☐ Decision documented (REST over GraphQL + rationale)
4. ☐ Action items with owners and dates
5. ☐ Next meeting noted

☐ **Compare:** Run the same prompt WITHOUT the skill (just paste the text normally). Notice the difference in consistency and structure.

Exercise 2: Add Supporting Files

Duration: 10 minutes

Task 1: Create a Template

Create `.claude/skills/meeting-notes/template.md` :

```
# Meeting Summary: [Title]

**Date:** [Date]
**Attendees:** [Names]
**Purpose:** [One-line purpose]

---

## Discussion Points

1. **[Topic 1]:** [1-2 sentence summary]
2. **[Topic 2]:** [1-2 sentence summary]
3. **[Topic 3]:** [1-2 sentence summary]

## Decisions

| Decision | Rationale | Decided By |
|-----|-----|-----|
| [Decision] | [Why] | [Who] |

## Action Items

| # | Action | Owner | Due Date | Status |
|---|-----|-----|-----|-----|
| 1 | [Task] | **[Name]** | [Date] | ☐ Open |

## Next Meeting

**Date:** [Date/Time]
```

```
**Agenda:** [Topics to cover]
```

```
---
```

```
*Generated by meeting-notes skill · [Timestamp]*
```

Task 2: Update SKILL.md to Reference the Template

Add this line to the Instructions section of your SKILL.md:

```
2. **Format as structured markdown:**  
  - Use the template in [template.md](template.md)
```

Task 3: Test with Template

Run the skill again with the same input. The output should now match the template format exactly — table for decisions, table for action items, next meeting section.

Exercise 3: Invocation Control & Advanced Features

Duration: 10 minutes

Task 1: Create a Deploy Skill (Manual Only)

Create `.claude/skills/deploy-check/SKILL.md` :

```
---  
name: deploy-check  
description: Pre-deployment checklist and verification  
disable-model-invocation: true  
allowed-tools: Bash(npm test) Bash(npm run build) Read Grep  
---  
  
# Pre-Deploy Checklist
```

Before deploying, verify:

1. Run ``npm test`` – all tests must pass
2. Run ``npm run build`` – no build errors
3. Check for console.log statements: ``grep -r "console.log" src/``
4. Verify no TODO comments: ``grep -r "TODO" src/``
5. Check package.json version was bumped

Report results as a checklist with ☐ or x for each item.
If ANY item fails, DO NOT proceed – report the failures.

Key points: - `disable-model-invocation: true` — Claude won't run this automatically - `allowed-tools` — restricts to test, build, and read-only operations - You must type `/deploy-check` to trigger it

Task 2: Create a Background Knowledge Skill

Create `.claude/skills/project-conventions/SKILL.md` :

```
---
name: project-conventions
description: Coding conventions and patterns for this project. Applies to all code.
user-invocable: false
---

# Project Conventions

## API Patterns
- All endpoints return `{ data, error, meta }` envelope
- Use HTTP status codes correctly (don't return 200 with error body)
- Validate request bodies with Zod schemas

## Code Style
- Prefer named exports over default exports
- Use async/await, never raw Promises
```

- Error messages must be user-facing friendly (no stack traces)

Database

- All queries use parameterized values (never string concatenation)
- Include `created\_at` and `updated\_at` on every table
- Soft delete with `deleted\_at` column, never hard delete

Key point: `user-invocable: false` means this won't appear in your `/` menu. Claude loads it automatically when you're working on code that matches the description.

Task 3: Install a Marketplace Skill

Try installing a skill from the community:

```
npx skills list
npx skills add vercel-labs/agent-skills@frontend-design
```

Browse the installed skill's SKILL.md to see how professional skills are structured.

Wrap-Up

What You Built

Skill	Type	Invocation
meeting-notes	Task + Template	You or Claude
deploy-check	Workflow	You only
project-conventions	Background knowledge	Claude only

Key Takeaways

1. **Skills = commands + context + templates.** More powerful than simple commands.

2. **Descriptions matter.** Good triggers = Claude loads it when needed. Bad triggers = never loads or loads on everything.
3. **Invocation control is critical.** Side effects → manual only. Domain knowledge → Claude only.
4. **Templates enforce consistency.** Same input → same output format, every time.
5. **Progressive disclosure saves tokens.** Supporting files load only when needed.

The Rule

If you've explained the same process to Claude 3 times → make it a skill.

Skills encode your expertise. Once built, they work for you, your team, and your agents — forever.

Lab complete. You now have three production-ready skills in your project.